Отчёт по практической работе

Создание CRUD-приложения. Модели ответов и обработка ошибок

Выполнил: Егор.

Дата выполнения: 17.10.2026.

Дисциплина: «Сетевые технологии».

Основание: методичка «ПЗ№4-1.pdf». Внутри PDF работа названа «Практическое занятие №5»; в исходном проекте сохранено название «Практическая работа №4».

1. Цель работы

Модернизировать приложение FastAPI для полного набора операций CRUD, сформировать ответы с помощью Pydantic и реализовать обработку ошибок с корректными HTTP-кодами. Персонализировать прикладные сообщения для разработчика Егора и проверить работу API реальными запросами curl.

2. Среда и запуск

Среда фактической проверки: Linux, Python 3.14.4; fastapi 0.143.3, pydantic 2.14.1, uvicorn 0.54.0. Зависимости установлены в локальное виртуальное окружение. Точный состав окружения сохранён в results/requirements-tested.txt.

Сервер и клиент запускаются в разных терминалах, как требуется методичкой. Для проверки выбран порт 8001: на порту 8000 уже работала исходная версия приложения. Все приведённые ниже результаты относятся к обновлённой версии на 8001.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
uvicorn api:app --host 127.0.0.1 --port 8001
```

Во втором терминале:

```
python3 verify_curl.py http://127.0.0.1:8001
```

Сценарий вызывает установленную программу curl и проверяет HTTP-коды и содержимое JSON. Перед проверкой он очищает список задач. Swagger UI доступен по адресу http://127.0.0.1:8001/docs, схема — http://127.0.0.1:8001/openapi.json.

3. Выполненная модернизация

В model.py добавлены TodoItem для изменения текста задачи и TodoItems для списка задач без идентификаторов. Модель Todo по-прежнему содержит id и item. Примеры схем заданы через ConfigDict(json_schema_extra=...), поскольку установлен Pydantic 2; это современный эквивалент Config.schema_extra из методички.

```
class TodoItem(BaseModel):
    item: str

class TodoItems(BaseModel):
    todos: list[TodoItem]
```

В todo.py добавлены PUT /todo/{todo_id}, DELETE /todo/{todo_id} и DELETE /todo. Создание задачи возвращает 201 Created. Успешные чтение, обновление и удаление возвращают 200 OK. PUT меняет

только item, сохраняя id.

```
@todo_router.get("/todo", response_model=TodoItems)
async def retrieve_todos() -> dict:
    return {"todos": todo_list}
```

Модель ответа фильтрует id только при выдаче общего списка. GET /todo/{todo_id} возвращает полную задачу с идентификатором.

Для отсутствующей задачи при GET, PUT и DELETE генерируется HTTPException с кодом 404. Прикладные сообщения содержат имя Егора. Дополнительно предотвращено создание задач с повторным ID: такой запрос возвращает 409 Conflict, чтобы обращение по идентификатору было однозначным.

```
raise HTTPException(
    status_code=status.HTTP_404_NOT_FOUND,
    detail="Егор, задача с указанным ID не найдена.",
)
```

Неверный тип ID в пути и отсутствие обязательного поля item обрабатываются стандартной валидацией FastAPI/Pydantic: HTTP 422 с описанием в detail. Системные сообщения валидатора сохранены в стандартном формате.

4. Персональная тема и типовые действия (пункт 4 задания)

В качестве персональной темы принята тема «Планировщик учебных задач студента Егора». Отдельная тема пользователем не была указана. Типовые действия выражены следующими командами-декораторами FastAPI и реализованы в проекте:

Действие студента	Команда FastAPI	Тело запроса / результат
Добавить учебную задачу	@todo_router.post("/todo", status_code=201)	Вход: id и item; ответ: message
Посмотреть план занятий	@todo_router.get("/todo", response_model=TodoItems)	todos: список объектов только с item
Открыть задачу	@todo_router.get("/todo/{todo_id}")	todo: id и item
Исправить описание задания	@todo_router.put("/todo/{todo_id}")	Вход: item; ответ: message
Удалить выполненную задачу	@todo_router.delete("/todo/{todo_id}")	message
Очистить учебный план	@todo_router.delete("/todo")	message

Варианты ошибок для этой темы:

Ситуация	Код HTTP	Модель тела ошибки
Задача с данным ID отсутствует	404	{"detail": "Егор, задача с указанным ID не найдена."}
Попытка повторно занять ID	409	{"detail": "Егор, задача с указанным ID уже существует."}

Некорректный ID или нет item	422	{"detail": [описания ошибок валидации]}

5. Фактические результаты curl

Ниже приведены команды, фактические коды HTTP и JSON-тела ответов запущенного сервера. Все ответы имели Content-Type: application/json. Полные заголовки, включая HTTP-статус, date, server, content-length и content-type, сохранены в results/curl-transcript.txt. Структурированные результаты находятся в results/curl-results.json.

5.1. Приветствие

```
curl --include -X GET http://127.0.0.1:8001/ \
  -H 'accept: application/json'
```

Получено: HTTP 200.

```
{
  "message": "Добро пожаловать в приложение Егора!"
}
```

5.2. Очистка перед проверкой

```
curl --include -X DELETE http://127.0.0.1:8001/todo \
  -H 'accept: application/json'
```

Получено: HTTP 200.

```
{
  "message": "Егор, все задачи успешно удалены!"
}
```

5.3. Пустой список

```
curl --include -X GET http://127.0.0.1:8001/todo \
  -H 'accept: application/json'
```

Получено: HTTP 200.

```
{
  "todos": []
}
```

5.4. Создание задачи

```
curl --include -X POST http://127.0.0.1:8001/todo \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{"id": 1, "item": "Изучить модели ответов FastAPI"}'
```

Получено: HTTP 201.

```
{
  "message": "Егор, задача успешно добавлена!"
}
```

5.5. Список без ID

```
curl --include -X GET http://127.0.0.1:8001/todo \
  -H 'accept: application/json'
```

Получено: HTTP 200.

```
{
```

```
  "todos": [
    {
      "item": "Изучить модели ответов FastAPI"
    }
  ]
}
```

5.6. Одна задача с ID

```
curl --include -X GET http://127.0.0.1:8001/todo/1 \
  -H 'accept: application/json'
```

Получено: HTTP 200.

```
{
  "todo": {
    "id": 1,
    "item": "Изучить модели ответов FastAPI"
  }
}
```

5.7. Обновление задачи

```
curl --include -X PUT http://127.0.0.1:8001/todo/1 \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{"item": "Подготовить отчёт по практической работе"}'
```

Получено: HTTP 200.

```
{
  "message": "Егор, задача успешно обновлена!"
}
```

5.8. Проверка обновлённого текста и сохранения ID

```
curl --include -X GET http://127.0.0.1:8001/todo/1 \
  -H 'accept: application/json'
```

Получено: HTTP 200.

```
{
  "todo": {
    "id": 1,
    "item": "Подготовить отчёт по практической работе"
  }
}
```

5.9. Повторный ID

```
curl --include -X POST http://127.0.0.1:8001/todo \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{"id": 1, "item": "Дубликат"}'
```

Получено: HTTP 409.

```
{
  "detail": "Егор, задача с указанным ID уже существует."
}
```

5.10. Дубликат не изменил список

```
curl --include -X GET http://127.0.0.1:8001/todo \
  -H 'accept: application/json'
```

Получено: HTTP 200.

```
{
  "todos": [
    {
      "item": "Подготовить отчёт по практической работе"
    }
  ]
}
```

5.11. Удаление одной задачи

```
curl --include -X DELETE http://127.0.0.1:8001/todo/1 \
  -H 'accept: application/json'
```

Получено: HTTP 200.

```
{
  "message": "Егор, задача успешно удалена!"
}
```

5.12. Получение отсутствующей задачи

```
curl --include -X GET http://127.0.0.1:8001/todo/1 \
  -H 'accept: application/json'
```

Получено: HTTP 404.

```
{
  "detail": "Егор, задача с указанным ID не найдена."
}
```

5.13. Обновление отсутствующей задачи

```
curl --include -X PUT http://127.0.0.1:8001/todo/1 \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{"item": "Новый текст"}'
```

Получено: HTTP 404.

```
{
  "detail": "Егор, задача с указанным ID не найдена."
}
```

5.14. Удаление отсутствующей задачи

```
curl --include -X DELETE http://127.0.0.1:8001/todo/1 \
  -H 'accept: application/json'
```

Получено: HTTP 404.

```
{
  "detail": "Егор, задача с указанным ID не найдена."
}
```

5.15. Создание первой задачи для общей очистки

```
curl --include -X POST http://127.0.0.1:8001/todo \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{"id": 2, "item": "Изучить HTTPException"}'
```

Получено: HTTP 201.

```
{
```

```
  "message": "Егор, задача успешно добавлена!"
}
```

5.16. Создание второй задачи для общей очистки

```
curl --include -X POST http://127.0.0.1:8001/todo \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{"id": 3, "item": "Проверить API через curl"}'
```

Получено: HTTP 201.

```
{
  "message": "Егор, задача успешно добавлена!"
}
```

5.17. Две задачи без ID

```
curl --include -X GET http://127.0.0.1:8001/todo \
  -H 'accept: application/json'
```

Получено: HTTP 200.

```
{
  "todos": [
    {
      "item": "Изучить HTTPException"
    },
    {
      "item": "Проверить API через curl"
    }
  ]
}
```

5.18. Удаление всех задач

```
curl --include -X DELETE http://127.0.0.1:8001/todo \
  -H 'accept: application/json'
```

Получено: HTTP 200.

```
{
  "message": "Егор, все задачи успешно удалены!"
}
```

5.19. Список после удаления всех задач

```
curl --include -X GET http://127.0.0.1:8001/todo \
  -H 'accept: application/json'
```

Получено: HTTP 200.

```
{
  "todos": []
}
```

5.20. Неверный тип ID в пути

```
curl --include -X GET http://127.0.0.1:8001/todo/not-a-number \
  -H 'accept: application/json'
```

Получено: HTTP 422.

```
{
  "detail": [
    {
```

```
      "type": "int_parsing",
      "loc": [
        "path",
        "todo_id"
      ],
      "msg": "Input should be a valid integer, unable to parse string as an integer",
      "input": "not-a-number"
    }
  ]
}
```

5.21. Нет обязательного поля item при создании

```
curl --include -X POST http://127.0.0.1:8001/todo \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{"id": 4}'
```

Получено: HTTP 422.

```
{
  "detail": [
    {
      "type": "missing",
      "loc": [
        "body",
        "item"
      ],
      "msg": "Field required",
      "input": {
        "id": 4
      }
    }
  ]
}
```

5.22. Нет обязательного поля item при обновлении

```
curl --include -X PUT http://127.0.0.1:8001/todo/4 \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{}'
```

Получено: HTTP 422.

```
{
  "detail": [
    {
      "type": "missing",
      "loc": [
        "body",
        "item"
      ],
      "msg": "Field required",
      "input": {}
    }
  ]
}
```

5.23. Ошибочные запросы не добавили задачи

```
curl --include -X GET http://127.0.0.1:8001/todo \
  -H 'accept: application/json'
```

Получено: HTTP 200.

```
{
  "todos": []
```

```
}
```

6. Результаты проверки и вывод

Успешно проверены 23 запросов curl. Подтверждены создание задачи с кодом 201, чтение списка без ID, чтение одной задачи с ID, изменение текста без изменения ID, удаление одной и всех задач. Все три операции над отсутствующей задачей возвращают 404. Повторный ID возвращает 409 и не меняет список. Некорректные запросы возвращают 422 и не добавляют данные.

Дополнительно через curl получена схема OpenAPI: проверена ссылка GET /todo на TodoItems, документирование ответа 201 для POST и наличие примера TodoItem. Копия схемы находится в results/openapi.json.

Задачи хранятся в памяти одного процесса, поэтому после перезапуска сервера список очищается. Для учебной демонстрации используется один worker. Dockerfile сохранён; проверка выполнена локально через Uvicorn, сборка Docker не проводилась.

Приложение реализует требования методички по CRUD, моделям ответов, обработке ошибок и персонализации сообщений. Исходные файлы api.py, model.py, todo.py и requirements.txt находятся рядом с отчётом; README.md содержит команды запуска и таблицу маршрутов.